



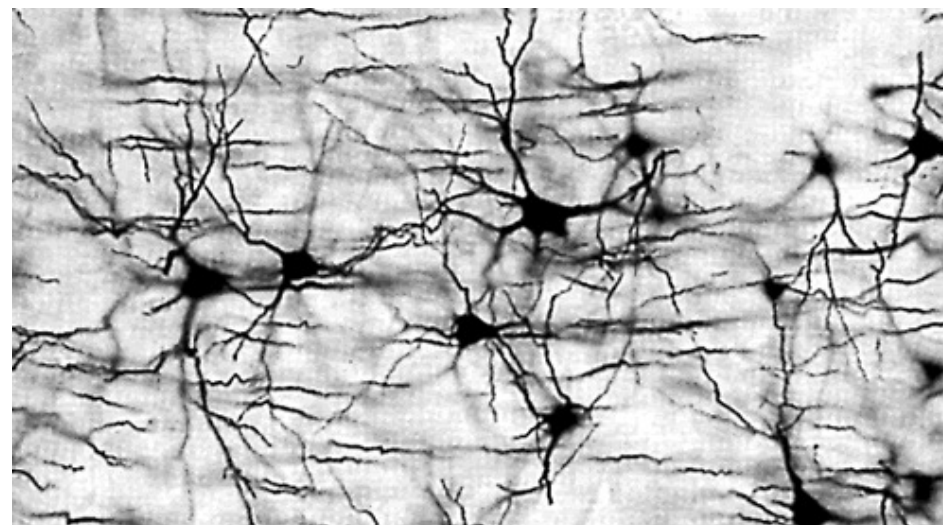
02456 – Week 3

Tricks of The Trade

Jes Frellsen

Technical University of Denmark

08 September 2025



Menu of the day

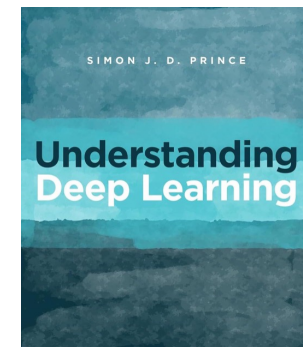
- Week 1: Neural nets
- Week 2: Learning
- **Week 3: Tricks of The Trade**
- Week 4: CNNs
- Week 5: RNNs
- Week 6: Transformers
- Week 7: Unsupervised
- Week 8: Mini-project

Lecture

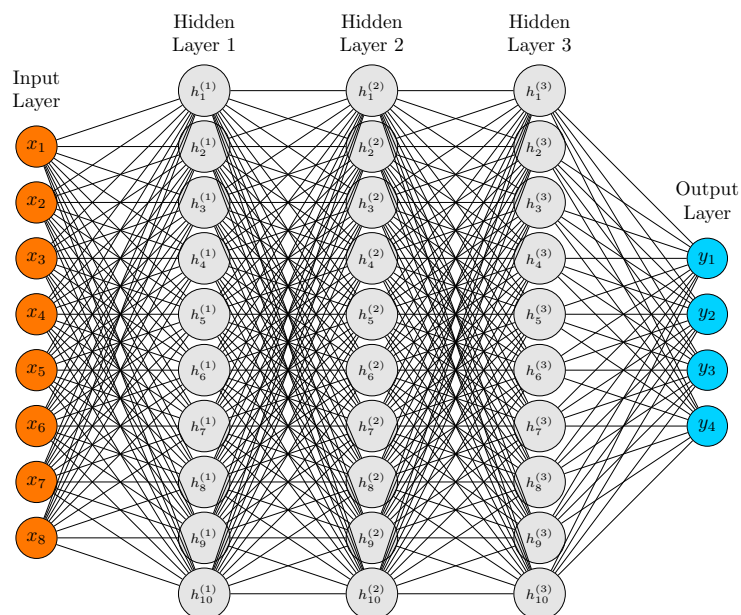
- Measuring performance (ch 8)
- Regularisation (ch 9)
- Residual networks (ch 11)

Exercises

- **Notebooks 3.1-3.4:**
 - FFNNs in **PyTorch**!
- **Problems:** 9.4, 11.4



Recap: Neural networks and learning



We can write the **output of layer ℓ** as

$$f^{(\ell)}(\mathbf{h}) = \sigma(W^{(\ell)}\mathbf{h} + \mathbf{b}^{(\ell)}),$$

where σ is a non-linear **activation function**.

The **joint function** is then

$$f_{\phi}(\mathbf{x}) = \mathbf{y} = f^{(4)}\left(f^{(3)}\left(f^{(2)}\left(f^{(1)}(\mathbf{x})\right)\right)\right)$$

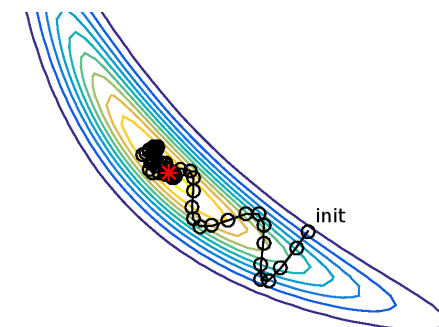
A **loss function** $L(\phi)$ captures the **mismatch** of $f_{\phi}(\mathbf{x}_i)$ and y_i

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} L(\phi)$$

Use the **negative log-likelihood**

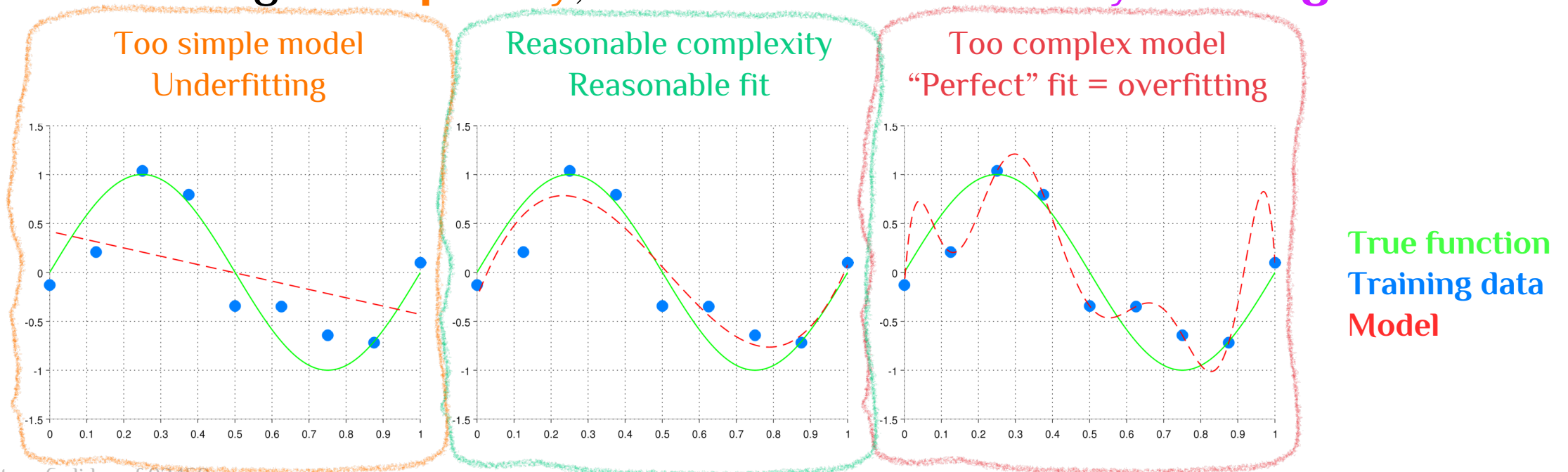
$$L(\phi) = -\sum_i \log p(y_i | f_{\phi}(\mathbf{x}_i))$$

- Use **gradient descent** to find $\hat{\phi}$
- **Backpropagation** gets gradients $\nabla_{\phi} L(\phi)$
- **Initialisation** is important



Neural network complexity

- **Complexity** is controlled by **number of hidden nodes**
 - More **hidden nodes** → more **complex model**
 - By **widening** or **adding** hidden layers
- With enough **complexity**, a neural net can **fit any training data**

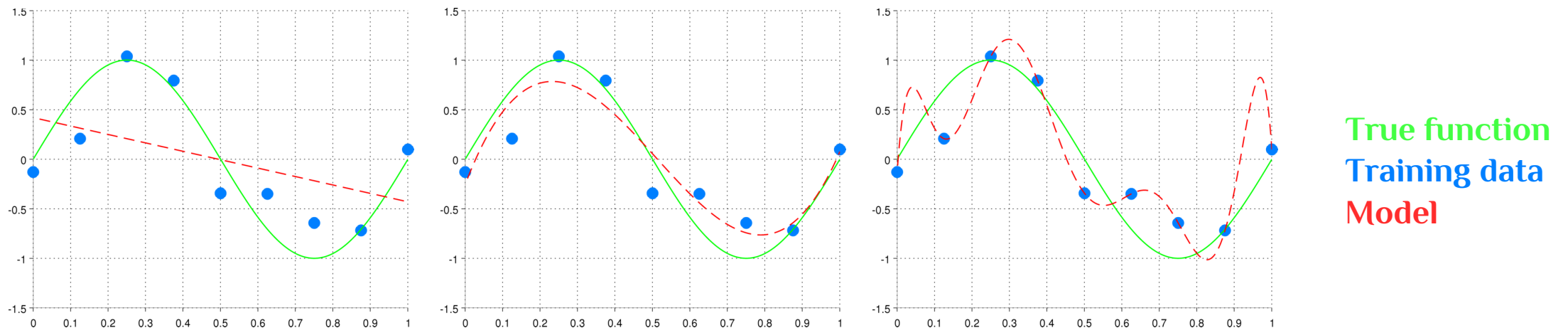


- How do we **measure the performance** of a model?
- How do we **select the right model complexity**?

We want the model **closes to the true data generating distribution**

Generalisation error

That model will have the **lowest error on unseen data**



Generalisation error

We have been writing, $L(\phi)$,
now explicit y and f_ϕ dependence

- We assume that there is an underlying **data generating distribution**
 $(\mathbf{x}_i, y_i) \sim p_D(\mathbf{x}, y)$

- For loss L , the **generalisation error** is given by

$$E^{\text{gen}} = \mathbb{E}_{\mathbf{x}, y \sim p_D(\mathbf{x}, y)} [L(f_\phi(\mathbf{x}), y)] = \int L(f_\phi(\mathbf{x}), y) p_D(\mathbf{x}, y) d\mathbf{x} dy$$

- We cannot calculate E^{gen} , but we can get a Monte Carlo estimate

$$E^{\text{gen}} \approx \frac{1}{n} \sum_{i=1}^n L(f_\phi(\mathbf{x}_i), y_i)$$

where $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ are sampled from p_D

Generalisation error

How can I select the model minimising the generalisation error?

- Generally, we cannot evaluate E^{gen} exactly
- We can **estimate** E^{gen}
- We can **analyse** E^{gen}

Analysing E^{gen} : Bias-variance trade-off

- What is the **expected generalisation error**, if we train model $f_{\phi(\mathcal{D})}$ on datasets $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ assuming a squared loss

$$\mathbb{E}_{\mathcal{D}}[E^{\text{gen}}] = \mathbb{E}_{\mathbf{x} \sim p_D(\mathbf{x})} \left[\left(\bar{y}(\mathbf{x}) - \bar{f}_{\phi(\mathcal{D})}(\mathbf{x}) \right)^2 + \text{Var}_{\mathcal{D}}[f_{\phi(\mathcal{D})}(\mathbf{x})] + \text{Var}[y(\mathbf{x})] \right]$$

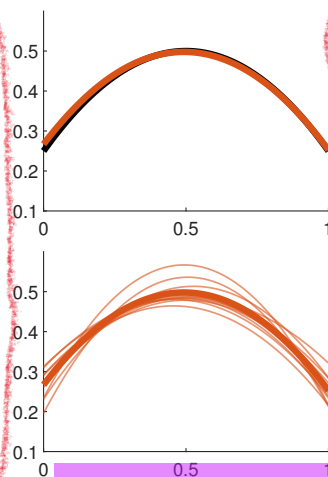
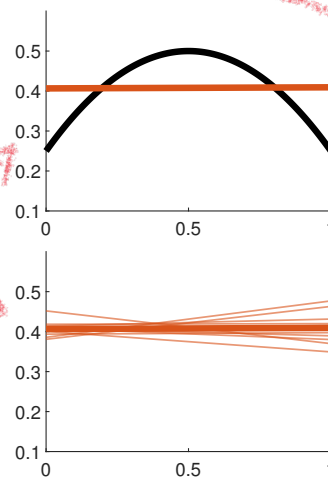
Bias: The average (over $\mathcal{D}$) model's deviation from the label mean

Variance: The variance of the model predictions over $\mathcal{D}$

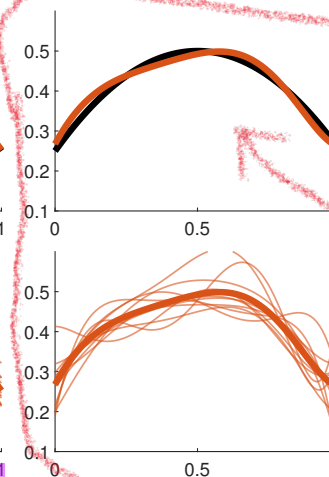
The **inherent variance** in the data distribution $p_D(y|\mathbf{x})$

Low model complexity

- High bias
- Low variance



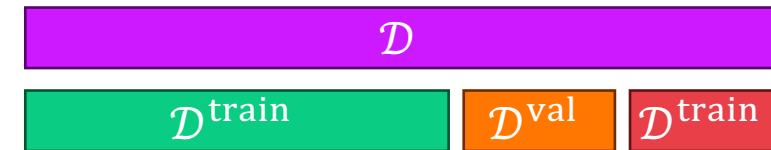
Sweet spot!



High model complexity

- Low bias
- High variance

Choosing hyperparameters and measuring performance



We **partition** our **dataset** into three: $\mathcal{D} = \mathcal{D}^{\text{train}} \cup \mathcal{D}^{\text{val}} \cup \mathcal{D}^{\text{test}}$

1. Train models $f_{\phi}^{(1)}, f_{\phi}^{(2)}, \dots$ with different hyperparameters on $\mathcal{D}^{\text{train}}$

$$\hat{\phi}^{(j)} = \underset{\phi}{\operatorname{argmin}} L(\phi, \mathcal{D}^{\text{train}})$$

- E.g., architectures, activations functions, learning rate, ...

Why is $E^{\text{train}} = L(\hat{\phi}, \mathcal{D}^{\text{train}})$ not a good estimate of E^{gen} ?

2. Evaluate the performance on the **validation set** $\mathcal{D}^{\text{val}}$

$$E^{\text{val},(j)} = L(\hat{\phi}^{(j)}, \mathcal{D}^{\text{val}})$$

Why is E^{val} not a good estimate of E^{gen} ?

3. Select the model with lowest validation error $\hat{j} = \underset{j}{\operatorname{argmin}} E^{\text{val},(j)}$

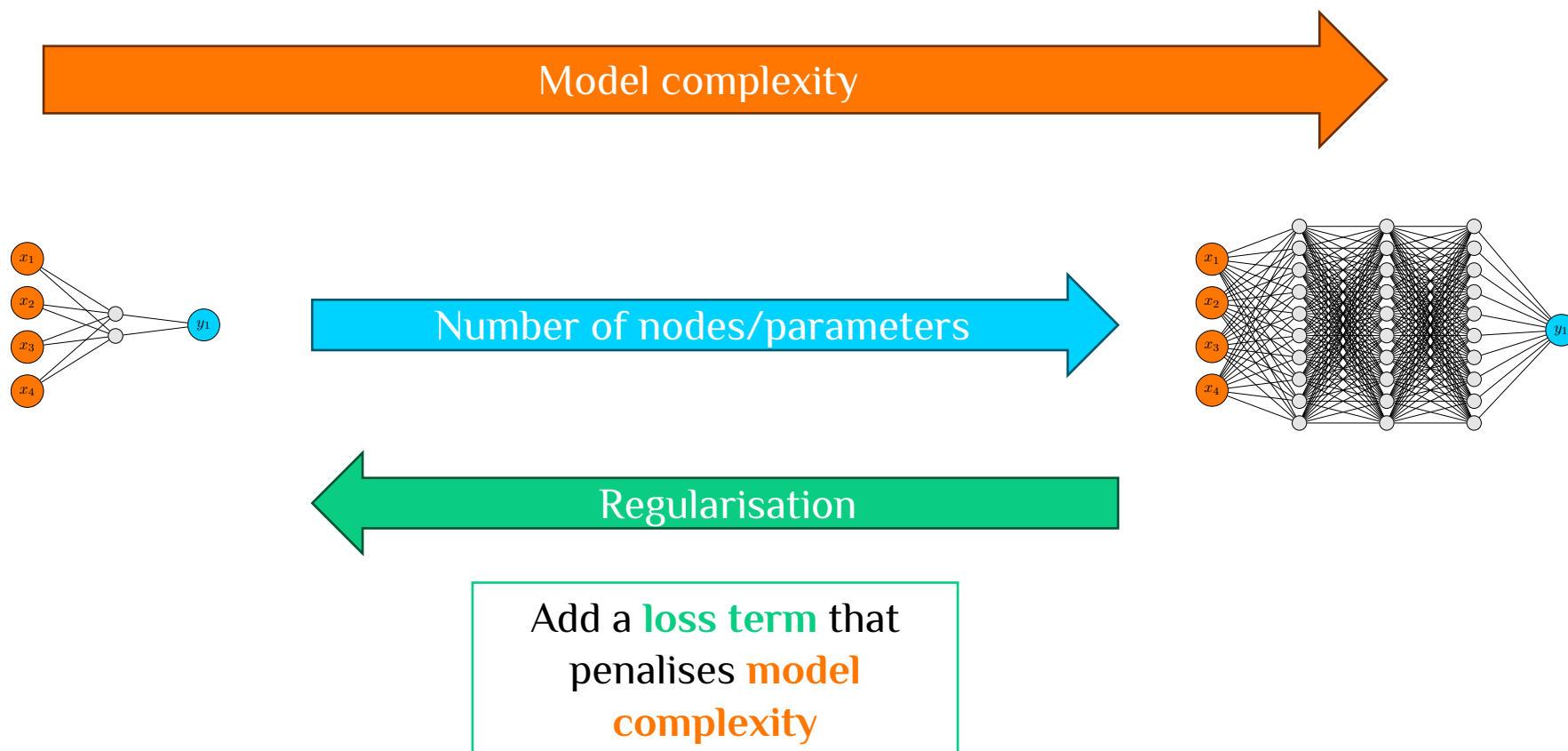
Biased estimators

- Both underestimates E^{gen}
- Due to **optimisation** E^{train} and selection on E^{val}

4. Estimate the **generalisation error** on the **test set**

$$E^{\text{gen}} \approx E^{\text{test}} = L(\hat{\phi}^{(\hat{j})}, \mathcal{D}^{\text{test}})$$

Model complexity



Explicit regularisation: weight decay

- Add a **penalty term** for **complexity** to the **loss**

$$L'(\phi) = L(\phi) + g(\phi)$$

- **Note:** $g(\phi)$ is only a function of **parameters** ϕ , not data.
- Only applied to **weights** and not biases

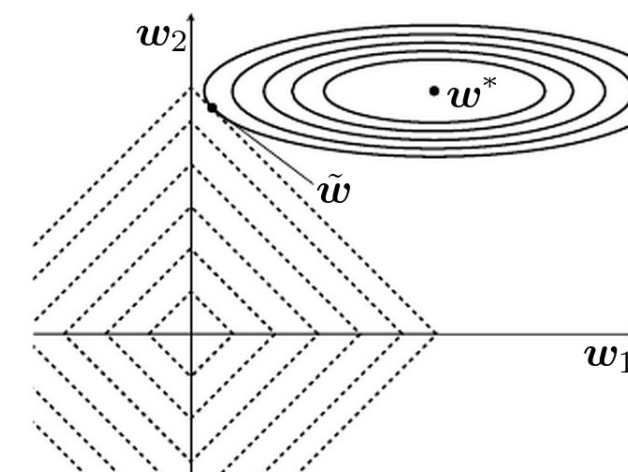
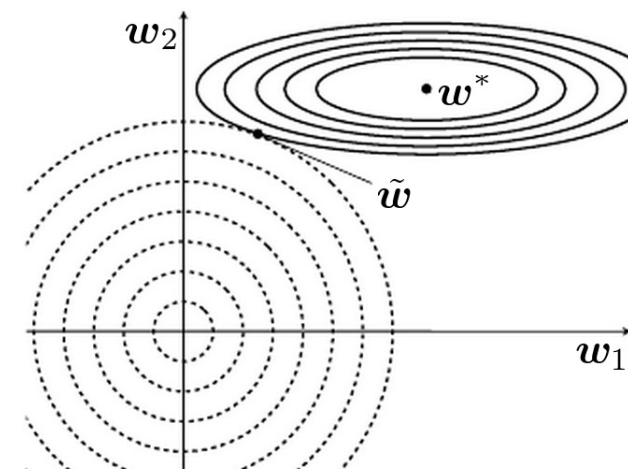
- **ℓ^2 -regularisation:** $g(\phi) = \frac{\lambda}{2} \|\phi\|^2 = \frac{\lambda}{2} \sum \phi_j^2$

- Hyperparameter λ controls strength
- **Gradient:** $\frac{\partial g(\phi)}{\partial \phi_j} = \lambda \phi_j$

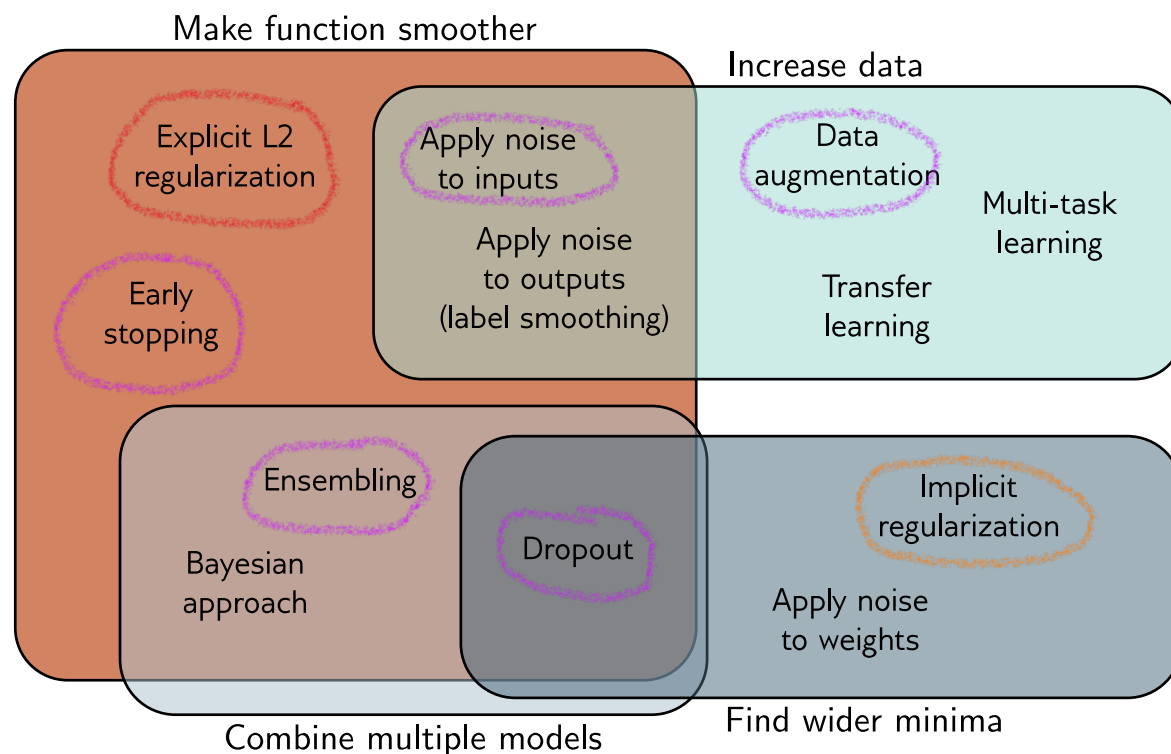
- **ℓ^1 -regularisation:** $g(\phi) = \frac{\lambda}{2} \|\phi\|_1 = \frac{\lambda}{2} \sum |\phi_j|$

- **Induces sparsity:** Often many params become zero.
- **Gradient:** $\frac{\partial g(\phi)}{\partial \phi_j} = \lambda \text{sign}(\phi_j)$

- In PyTorch: implemented though gradient in optimizer



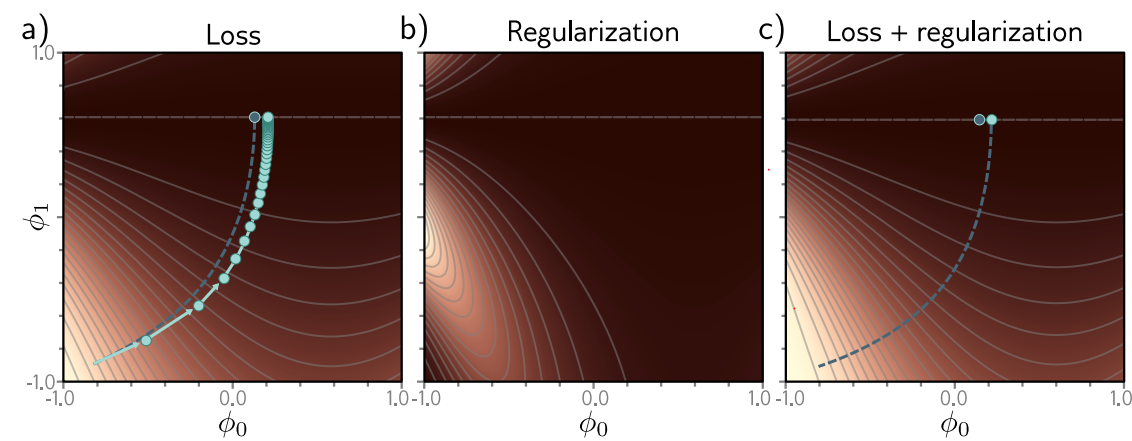
Tricks to improve performance



Implicit regularisation

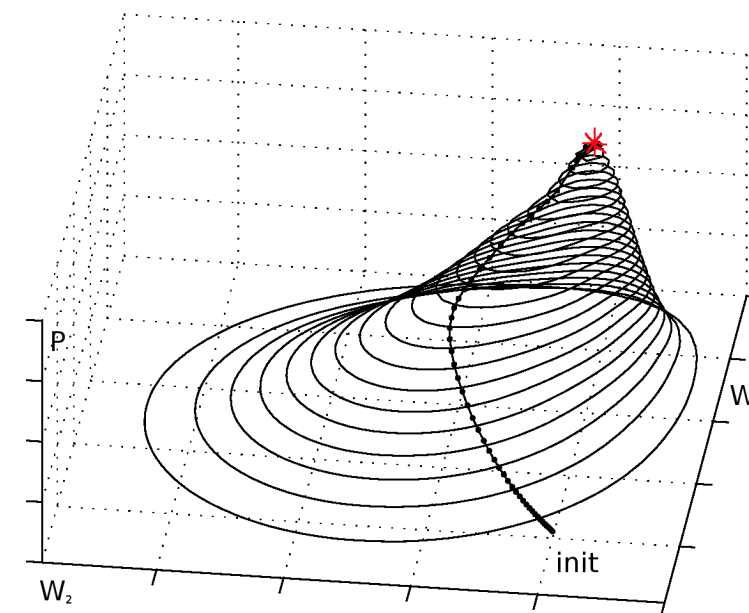
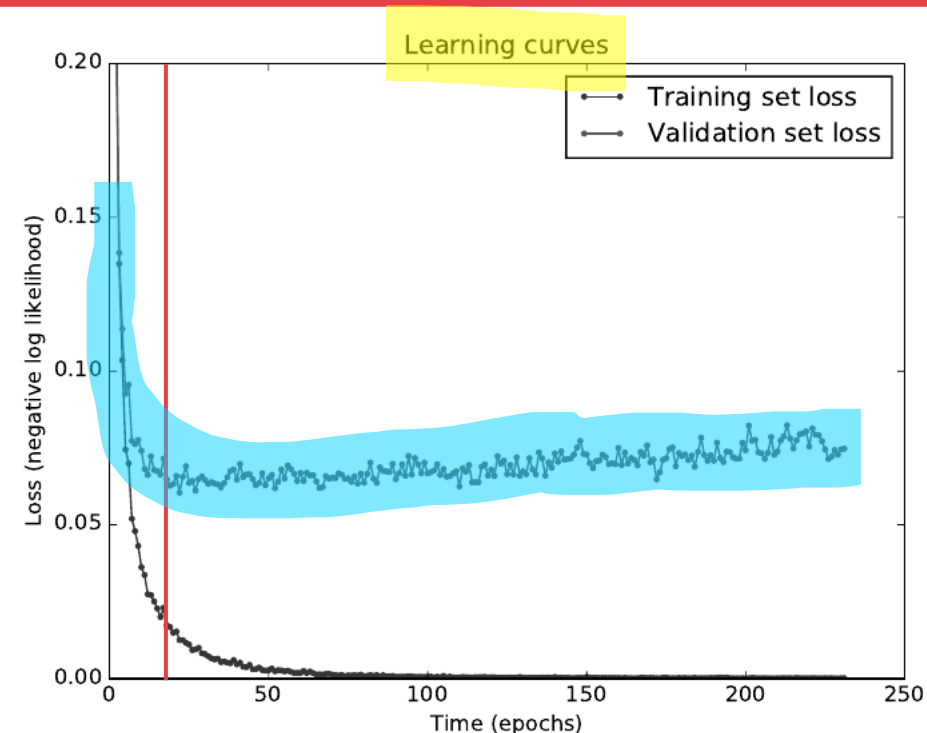
- **Gradient descent**
 - Penalizes large gradients
- **Stochastic gradient descent**
 - In addition, penalizes large variance gradients

Due to **finite step size**



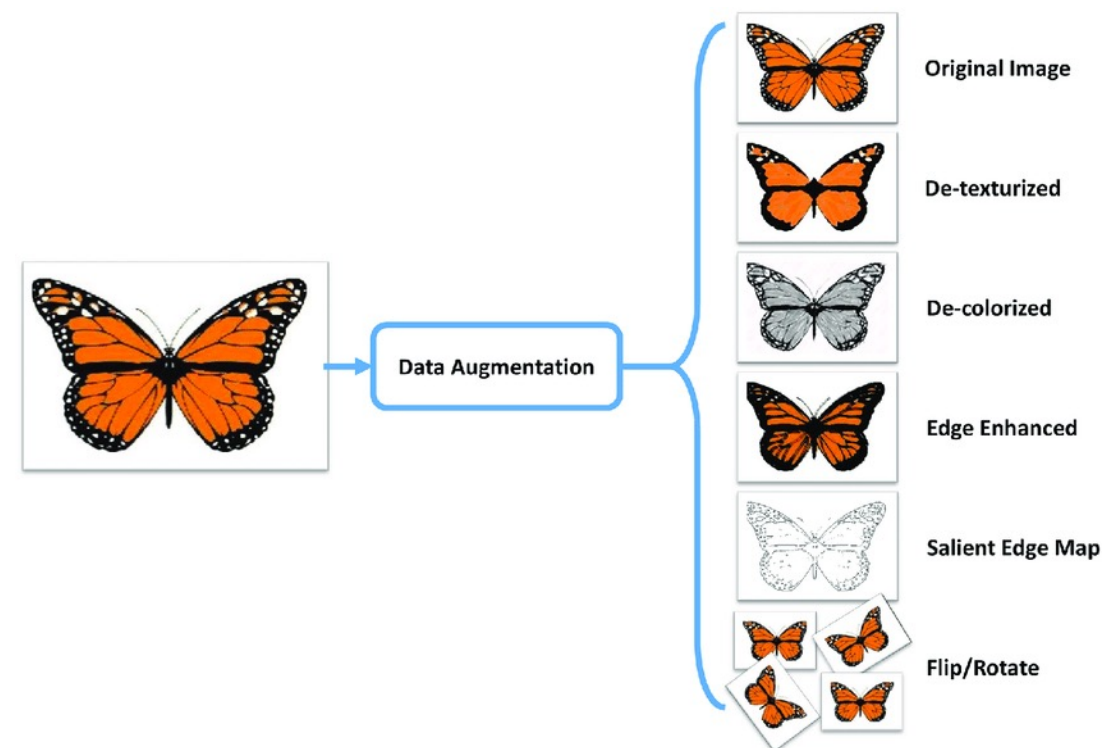
Early stopping

- Monitor **validation performance** during training
- Stop when it starts to **deteriorate**
 - Keeps solution close to the initialization
 - When combined with other regularisations, this might never happen



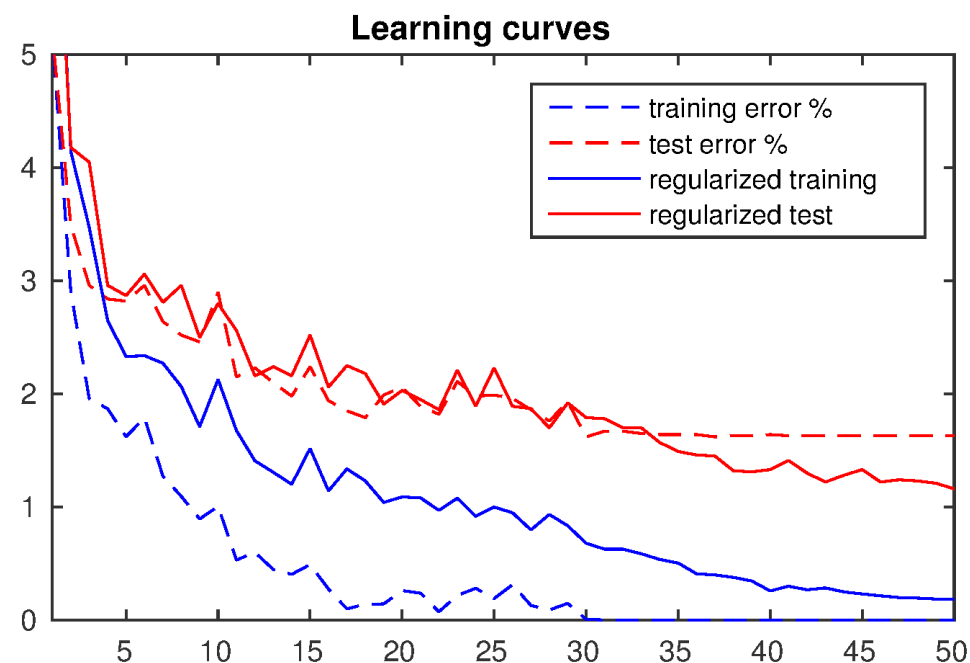
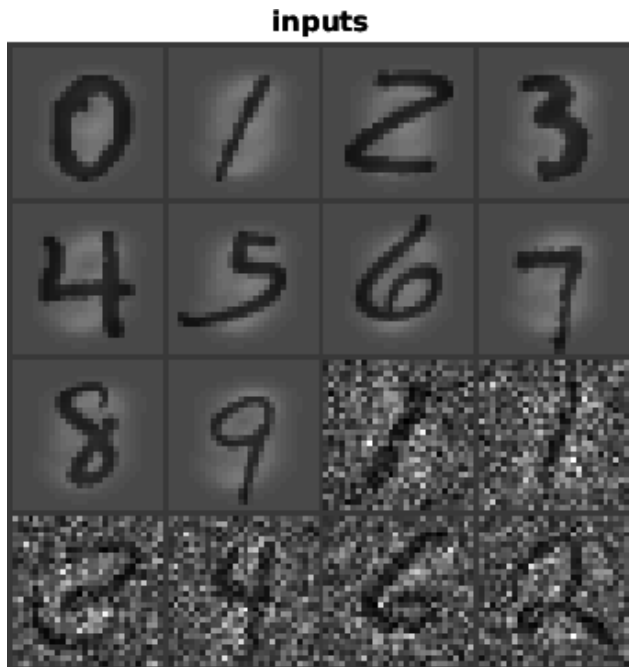
Data augmentation

- Augmented data by data-specific transformations:
 - E.g., cropping all combinations of 1-pixels edges in an image gives you 9 times the data!
 - The network can learn these invariances
 - **This can give you (nearly) infinite data!**



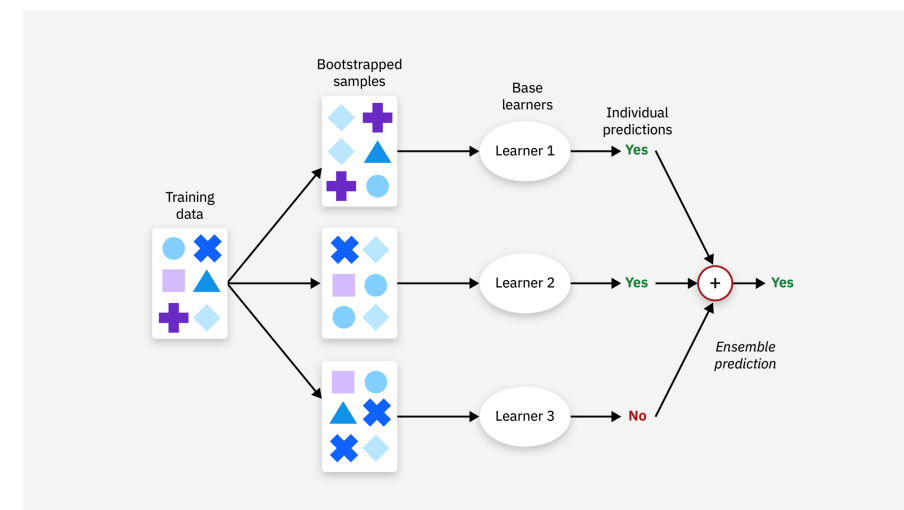
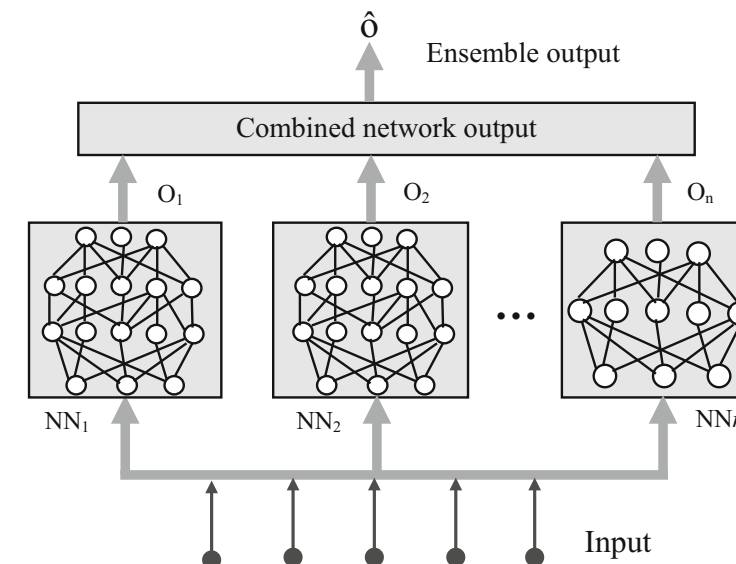
Injecting data noise

- Inject random **noise** during training (random in each epoch)
 - Can be applied to **input data**, to **hidden activations**, or to **weights**
 - Can be seen as **data augmentation**
 - **Simple and effective**



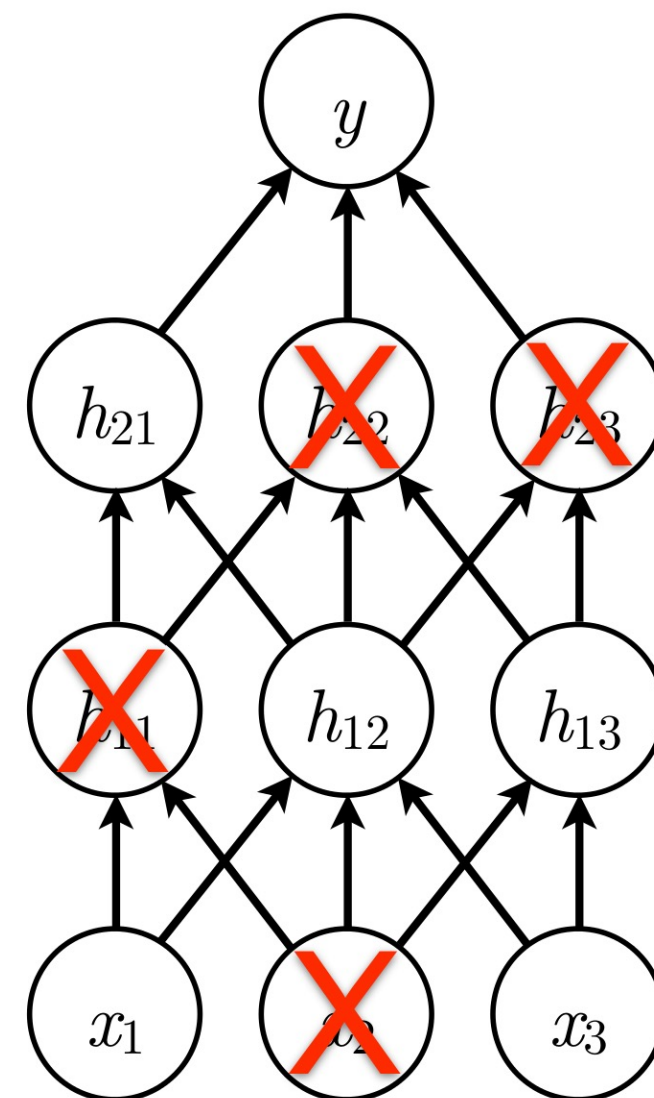
Ensemble methods

- Train **several models**
 - **Regression**: average outputs
 - **Classification**: average probabilities (or majority voting)
- **Known from random forest**:
 - Bagging trees + feature subsampling
 - Bagging: resampled data with replacement
- Make **individual models different** by
 - varying random seed
 - varying model architecture
 - varying hyperparameters and algorithms
 - varying data (dropping examples or dimensions)
- *This often improves results, especially if models are diverse!*

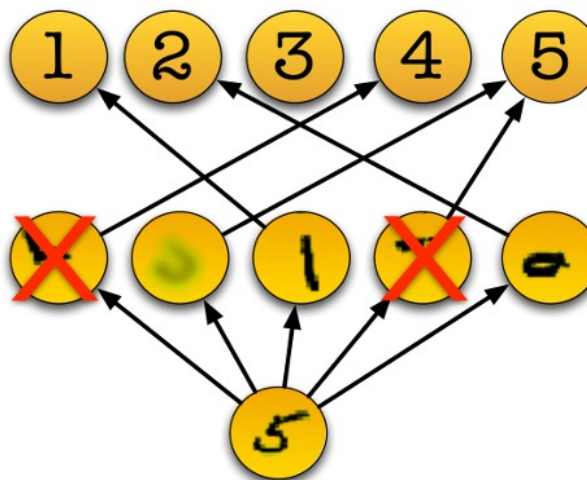
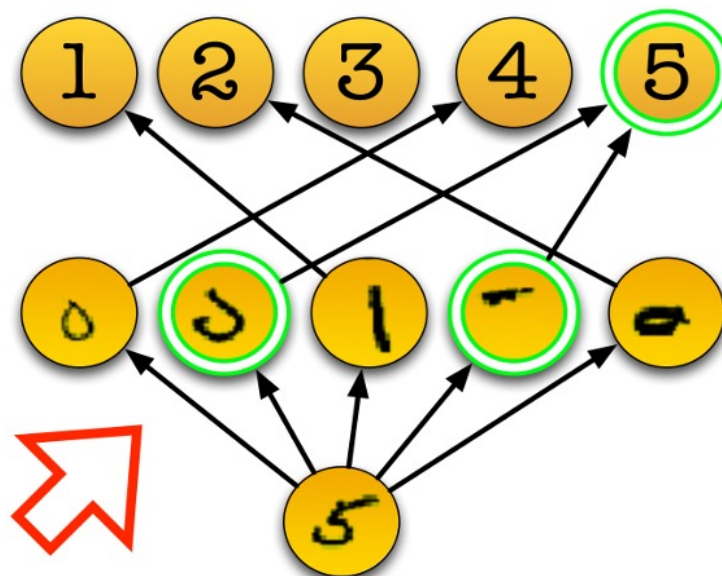
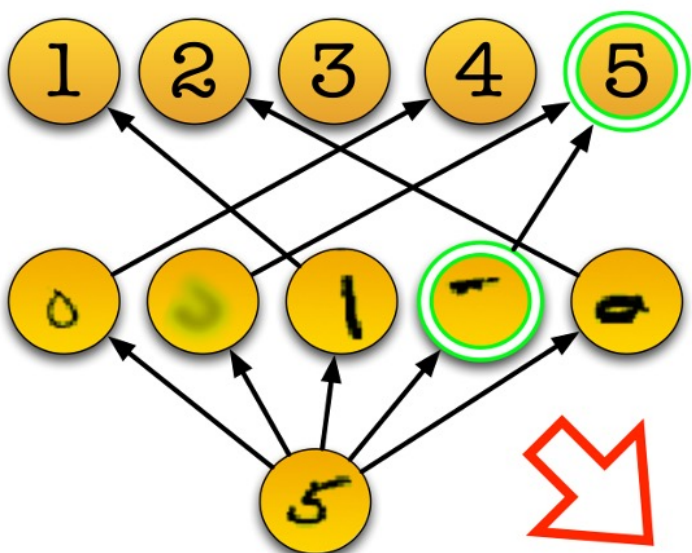


Dropout

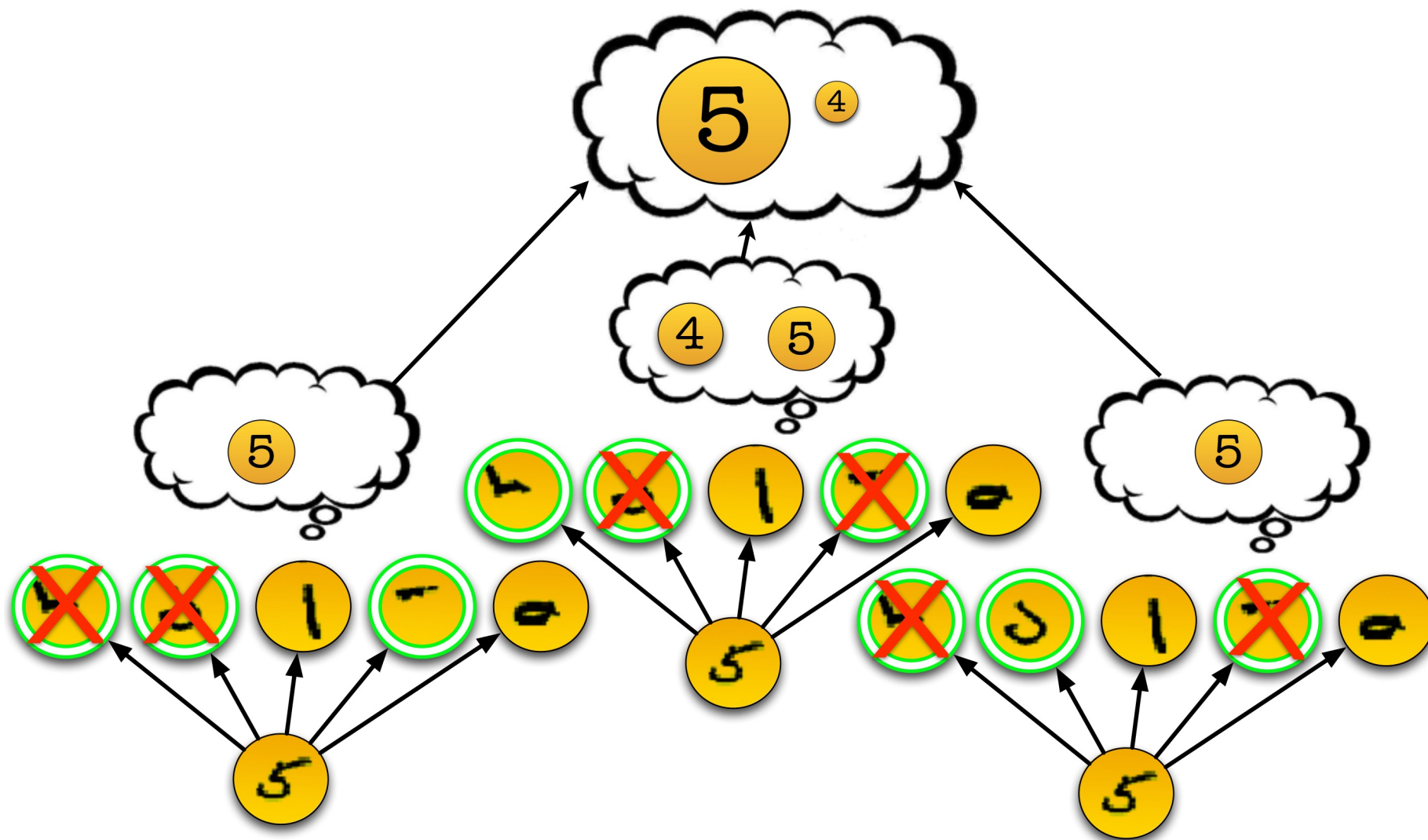
- Each time we present an input x , **randomly delete** (hidden) **nodes** with probability $\rho = 0.5$
- Can be seen as:
 - Injecting **multiplicative binary noise**
 - Training an **ensemble** of $2^{\text{\#hidden nodes}}$ networks with weight sharing
- At **test time**:
 - Use all nodes but multiply weights by ρ
 - Use an ensemble



Dropout training

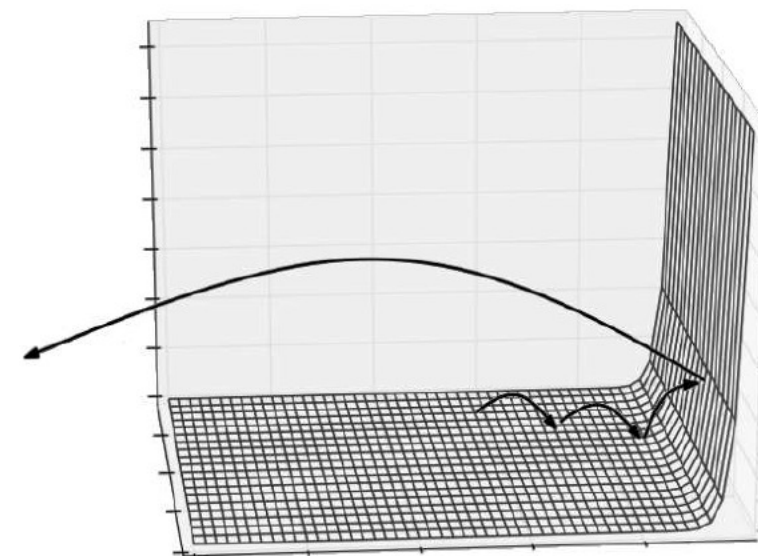


Dropout as ensemble prediction



Vanishing or exploding gradients

- We want to avoid
 - **Vanishing gradients**: $\left\| \frac{\partial L}{\partial W^{(\ell)}} \right\| \rightarrow 0$ for early layers ℓ
 - **Exploding gradients**: $\left\| \frac{\partial L}{\partial W^{(\ell)}} \right\| \rightarrow \infty$ for early layers ℓ
- **Weight initialisation** helps with this
- Highly **nonlinear model**:
 - Gradient update can **catapult parameters** very far
 - Heuristic: **Clip** the magnitude of the gradient



Residual networks

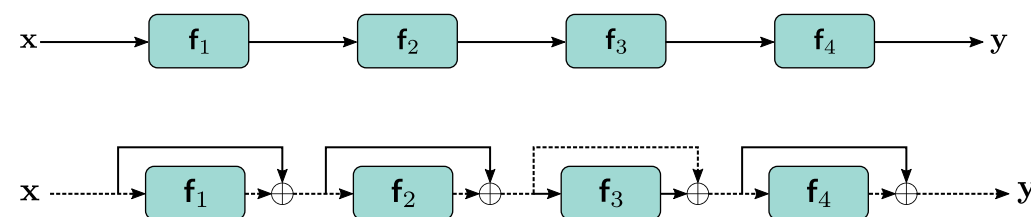
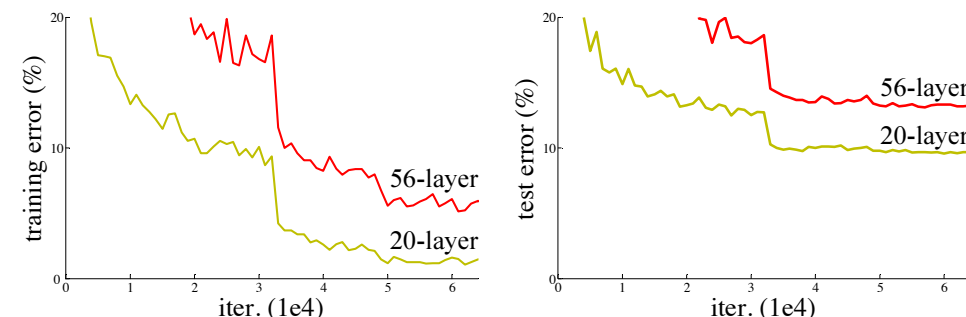
- We may still have problems with **gradient flow** in **very deep networks**
- Shattered (uncorrelated) gradients
- **Residual (skip) connections** help by adding an **identity path**:

$$\mathbf{h}^{(\ell)} = \mathbf{h}^{(\ell-1)} + f_{\phi_\ell}^{(\ell)}(\mathbf{h}^{(\ell-1)})$$

- Allows gradients to flow directly through the network

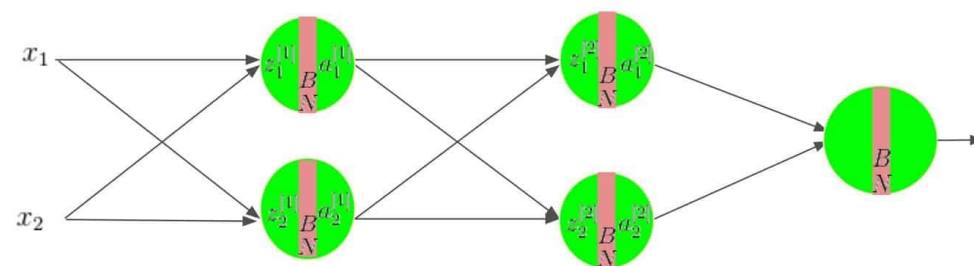
$$\frac{\partial \mathbf{h}^{(\ell)}}{\partial \mathbf{h}^{(\ell-1)}} = I + \frac{\partial f_{\phi_\ell}^{(\ell)}(\mathbf{h}^{(\ell-1)})}{\partial \mathbf{h}^{(\ell-1)}}$$

- *Makes training of very deep networks feasible*

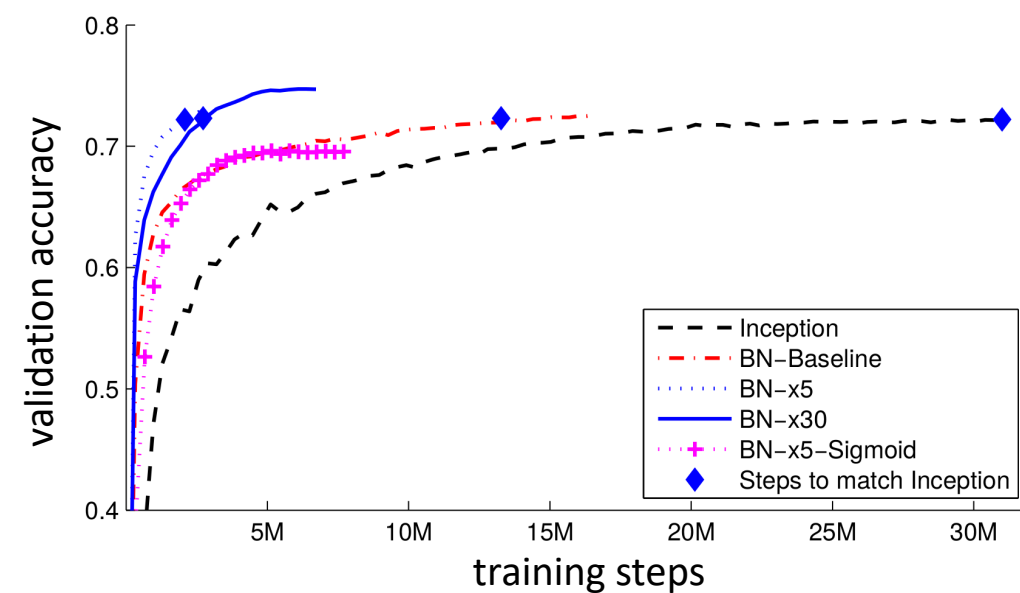


Batch normalisation

- Batch normalisation further improve learning
 - Keeps activations and gradients well-scaled
 - Improves learning
 - Allows bigger learning rates (5x and 30x)



$$z^{[l]} = W^{[l]} a^{[l-1]} \longrightarrow \begin{aligned} \mu^{[l]} &= \frac{1}{m} \sum_i z^{[l](i)} \\ \sigma^{[l]2} &= \frac{1}{m} \sum_i (z^{[l](i)} - \mu^{[l]})^2 \\ z_{norm}^{[l](i)} &= \frac{z^{[l](i)} - \mu^{[l]}}{\sqrt{\sigma^{[l]2} + \epsilon}} \\ \tilde{z}^{[l](i)} &= \gamma^{[l]} z_{norm}^{[l](i)} + \beta^{[l]} \end{aligned} \longrightarrow a^{[l]} = g^{[l]}(\tilde{z}^{[l]})$$



Today's exercises!

- Three Python notebooks ([Week3](#) [Feedforward](#) [PyTorch](#))
 - Introduction to **PyTorch** and **AutoDiff**
 - FFNNs on Half Moon and MNIST
- Two **problems** from the book on covered material

Thank you!